

گزارش سوال ۳ عملی - تمرین ۴ - سیستم عامل

در این تمرین یک ابزار مدیریت فایل سیستم ساده سازی شده در سطح کاربر با زبان C می‌سازیم. ساختار کلی به این صورت است که ابتدا با دستور `init filesystem.db` یک فایل باینری ۲۵۶ مگابایتی با نام `filesystem.db` (یا هر نام دیگر که در دستور تعیین شود) می‌سازیم و این فایل را به عنوان بخشی از دیسک که سعی در مدیریت آن داریم در نظر می‌گیریم. اگر چنین فایلی با نام مشخص شده وجود نداشت، آن را می‌سازیم:

```
• martin@Matin:~/Term5/OS/HW/3$ gcc -std=c11 -Wall cli.c fs.c -o myfs
○ martin@Matin:~/Term5/OS/HW/3$ ./myfs
Simple FS CLI (Step 1) - type 'help' for commands.
myfs> init filesystem.db
[DEBUG] init called with path: filesystem.db
Created new filesystem 'filesystem.db' (size = 256.00 MB)
myfs> █
```

و اگر وجود داشت، آن را باز کرده و تطابق magic number و version را بررسی می‌کنیم:

```
○ martin@Matin:~/Term5/OS/HW/3$ ./myfs
Simple FS CLI (Step 1) - type 'help' for commands.
myfs> init wrongversion.db
Error: unsupported FS version.
Failed to initialize filesystem.
myfs> █
```

در صورت صحت اعداد بالا، فایل مورد نظر را باز می‌کنیم:

```
○ martin@Matin:~/Term5/OS/HW/3$ ./myfs
Simple FS CLI (Step 1) - type 'help' for commands.
myfs> init filesystem.db
Loaded existing filesystem filesystem.db
myfs> █
```

در زیر نمایی کلی از دستورات پیاده سازی شده را داریم:

```
myfs> help
Available commands:
  init <path>
  open <filename> <flags>
  flags:
    0 -> open existing file
    1 -> create & open new file
  read <pos> <nbytes>
  write <pos> <nbytes>
  shrink <newsize>
  file_stats
  fs_stats
  rm
  close
  help
  exit
myfs> █
```

در ادامه به توضیح مختصر عملکرد دستورات می پردازیم. برای امتحان تمامی حالات ممکن، میتوانید فایل cli.c را اجرا کرده

و دستورات را با ورودی های مختلف و در توالی دلخواه امتحان کنید.

برای ایجاد فایل جدید از flag 1 و برای باز کردن فایل موجود، از flag 0 استفاده می کنیم. دستورات read, write

file_stats, close و shrink مطابق توضیحات دستورکار پیاده سازی شده اند:

```
matin@Matin:~/Term5/OS/HW/3$ ./myfs
myfs> open file1 1
File created and opened: file1
myfs> write 0 5
12345
Wrote 5 bytes at pos 0.
myfs> myfs> write 10 20
thereisahousebackin.
Wrote 20 bytes at pos 10.
myfs> myfs> read 2 23
345thereisahouseba
myfs> file_stats
File name: file1
Size: 30 bytes
First block: 4281
Next entry: 0
myfs> shrink 25
File truncated to 25 bytes.
myfs> file_stats
File name: file1
Size: 25 bytes
First block: 4281
Next entry: 0
myfs> read 0 60
12345thereisahouseba
```

```
myfs> open file1 1
File created and opened: file1
myfs> write 0 5
12345
Wrote 5 bytes at pos 0.
myfs> myfs> write 10 2
89
Wrote 2 bytes at pos 10.
myfs> myfs> read 0 50
1234589
myfs> file_stats
File name: file1
Size: 12 bytes
First block: 4180
Next entry: 0
myfs> close
File closed.
myfs> █
```

در زیر کارکرد دستور `fs_stats` و کاهش فضای استفاده شده پس از دستور `rm` را مشاهده می کنید:

```
o martin@Martin:~/Term5/OS/HW/3$ ./myfs
Simple FS CLI (Step 1) - type 'help' for commands.
myfs> init filesystem.db
Loaded existing filesystem filesystem.db
myfs> fs_stats
Filesystem statistics:
  Total size: 268435456 bytes (256.00 MB)
  Block size: 4096 bytes (4.00 KB)
  File count: 2
  Used space: 4316 bytes (4.21 KB)
  Free space: 268431140 bytes
myfs> open file1 0
File opened: file1 (size = 12 bytes)
myfs> rm
File removed.
myfs> open fiell 0
File opened: fiell (size = 25 bytes)
myfs> rm
File removed.
myfs> fs_stats
Filesystem statistics:
  Total size: 268435456 bytes (256.00 MB)
  Block size: 4096 bytes (4.00 KB)
  File count: 0
  Used space: 4096 bytes (4.00 KB)
  Free space: 268431360 bytes
myfs> █
```

ساختار کلی فایل سیستم به این صورت است که در ابتدای فایل سیستم، یک `superblock` با مشخصات خواسته شده

ایجاد می کنیم و در آن اطلاعات کلی درمورد وضعیت فایل سیستم را ذخیره می کنیم:

```
// ----- initialize fs -----
#define MAGIC_VAL 0xDEADBEEF
#define FS_VERSION 1

#define FS_TOTAL_SIZE (256ULL * 1024 * 1024) // 256 MB
#define SUPERBLOCK_SIZE 4096 // 4 MB
#define BLOCK_SIZE 4096

// Superblock structure
typedef struct {
    uint32_t magic; // must be 0xDEADBEEF
    uint32_t version; // FS version
    uint32_t block_size; // should be 4096
    uint32_t last_block; // last allocated block index
    uint32_t file_list_head; // offset of first file entry
    uint32_t file_count; // number of files
    uint32_t free_block_head; // head of free block list (0 = none)
    uint8_t reserved[SUPERBLOCK_SIZE - 7 * 4];
} superblock_t;
```

سپس برای هر فایل یک file entry ایجاد می‌کنیم که در آن مشخصات آن فایل و آدرس فایل بعدی (در linkedlist ذخیره شده است):

```
// ----- define file entries -----
#define FILENAME_MAX_LEN 64

typedef struct file_entry {
    char name[FILENAME_MAX_LEN]; // file name (no '/')
    uint32_t type;                // 0 = normal file (for now)
    uint32_t permission;         // ignore or set constant
    uint32_t size;               // file size in bytes

    // where file data begins (offset in the 256MB filesystem)
    uint32_t first_block;

    // linked list
    uint32_t next;               // offset of next file_entry (or 0)
} file_entry_t;
```